

SuperMath

Roteiro de Apresentação — Computação Numérica

Ciência da Computação · UNISAGRADO · 2025

Este documento cobre tudo que o professor pode perguntar: conceitos dos métodos, implementação no código, validação de entradas e diferenças entre algoritmos.

1 VISÃO GERAL DO PROJETO

O SuperMath é uma aplicação web de métodos numéricos em Python com Streamlit. Interface interativa para executar e visualizar algoritmos clássicos de cálculo numérico, sem instalação local.

Python	Linguagem base de todo o sistema
Streamlit	Converte o script Python em aplicação web com interface gráfica
NumPy / SciPy	Operações matriciais e cálculos numéricos
SymPy	Cálculo simbólico — deriva funções automaticamente (Newton-Raphson)
Matplotlib	Geração dos gráficos de resultado
Groq API	Serviço de inferência que roda o modelo de IA (Llama 3.3 70B)

Estrutura de arquivos: `app.py` — interface principal; `tools.py` — função de leitura de sessão usada pelo Clark Calc; `metodos_numericos/` — pacote com os algoritmos divididos em subpastas `raizes/`·`sistemas/`·`ajustes/`.

2 MÓDULO — RAÍZES DE FUNÇÕES

Objetivo: encontrar x tal que $f(x) = 0$.

Como o sistema interpreta a função

O texto digitado passa pelo SymPy, que o converte em expressão simbólica. Sintaxe inválida gera erro antes de calcular. Para Newton-Raphson, o SymPy deriva automaticamente.

2.1 Bisseção

Divide $[a,b]$ ao meio repetidamente, ficando com o subintervalo de troca de sinal.

a e b	Extremos do intervalo
Tolerância	Tamanho mínimo do intervalo para parar
Máx. iterações	Limite de divisões

2.2 Ponto Fixo

Itera $x_{n+1} = g(x_n)$ até $|x_{n+1} - x_n| < \text{tolerância}$.

$g(x)$	Função de iteração (usuário fornece)
x_0	Chute inicial
Tolerância	Diferença mínima entre iterações

2.3 Newton-Raphson

Usa $x_{n+1} = x_n - f(x_n)/f'(x_n)$. Derivada calculada pelo SymPy.

x_0	Chute inicial
Tolerância	Diferença mínima entre iterações

2.4 Secantes

Aproxima $f'(x) \approx (f(x_1) - f(x_0))/(x_1 - x_0)$. Dispensa derivada analítica.

x_0 e x_1	Dois chutes iniciais distintos

Tolerância	Diferença mínima entre iterações
------------	----------------------------------

2.5 Comparativo

Bisseção	Lenta (linear)	Não	Sim, se intervalo válido
Ponto Fixo	Moderada	Não	Depende de $g(x)$
Newton-Raphson	Rápida (quadrática)	Sim (auto)	Depende de x^*
Secantes	Moderada-rápida	Não	Depende de x^* e x^{**}

3 MÓDULO — SISTEMAS LINEARES

Objetivo: resolver $Ax = b$.

3.1 Eliminação de Gauss

Direto. Triangulariza A por operações de linha com pivoteamento parcial, resolve por substituição retroativa.

3.2 Fatoração LU

Direto. Decompõe $A = P \cdot L \cdot U$. Resolve $Ly = Pb$ e depois $Ux = y$. Decomposição reutilizável para múltiplos vetores b .

3.3 Jacobi

Iterativo. Atualiza todas as variáveis simultaneamente com valores da iteração anterior. Requer dominância diagonal.

3.4 Gauss-Seidel

Iterativo. Atualiza cada variável imediatamente com valores mais recentes. Geralmente converge mais rápido que Jacobi.

Gauss	Direto	Caso geral, uso único
LU	Direto	Múltiplos vetores b
Jacobi	Iterativo	Sistemas grandes e esparsos
Gauss-Seidel	Iterativo	Quando Jacobi converge lento

4 MÓDULO — AJUSTE DE CURVAS

Objetivo: encontrar a curva que melhor representa pontos (x,y) pelo critério dos mínimos quadrados.

4.1 Regressão Linear

Ajusta $y = a \cdot x + b$. Coeficientes por fórmula fechada: $a = (n \cdot \sum xy - \sum x \cdot \sum y) / (n \cdot \sum x^2 - (\sum x)^2)$.

4.2 Mínimos Quadrados Polinomial

Ajusta polinômio de grau n com `np.polyfit`. Grau maior = mais flexível, mas risco de overfitting.

■ Regra prática: grau bem menor que o número de pontos. Grau 2–3 basta para a maioria dos dados.

5 ASSISTENTE CLARK CALC

Llama 3.3 70B via Groq. Usa **function calling** para ler a sessão em tempo real. Aciona pelo botão "Como funciona?" ou pergunta livre na barra lateral.

Limitações: requer internet e GROQ_API_KEY. Não executa cálculos.

6 PERGUNTAS QUE O PROFESSOR PODE FAZER

■ Onde estão implementados os métodos no código?

Cada método está em seu próprio arquivo dentro de `metodos_numericos/`:

- Raízes: `raizes/bissecao.py` · `raizes/ponto_fixo.py` · `raizes/newton_raphson.py` · `raizes/secantes.py`
- Sistemas: `sistemas/gauss_pivoteamento.py` · `sistemas/fatoracao_lu.py` · `sistemas/jacobi.py` · `sistemas/gauss_seidel.py`
- Ajustes: `ajustes/regressao_linear.py` · `ajustes/minimos_quadrados.py`

■ Para que serve o arquivo `tools.py`?

O `tools.py` contém a função `obter_ultimo_calculo(st_session_state)`, que serializa para JSON o estado do último cálculo armazenado na sessão do Streamlit. Essa função é passada ao modelo Llama como uma tool (function calling) — quando o Clark Calc precisa analisar os dados da tela, ele chama essa função e recebe o resultado em JSON.

■ Como o `app.py` se conecta aos métodos numéricos?

O `app.py` importa cada função diretamente: `from metodos_numericos.raizes.bissecao import bissecao` e assim por diante para todos os 10 métodos. Quando o usuário clica em CALCULAR, o `app.py` chama a função correspondente, guarda o resultado em `st.session_state['ultimo_calculo']` e exibe na tela.

■ O que é `st.session_state` e para que serve no projeto?

`st.session_state` é um dicionário persistente do Streamlit que mantém dados entre rerenderizações da página — como o resultado do último cálculo. No SuperMath, ele armazena `'ultimo_calculo'` com o método, parâmetros, resultado e tabela de iterações. O Clark Calc acessa esse estado via `tools.py` para analisar o que está na tela.

■ Como o SymPy é usado para derivar funções automaticamente no Newton-Raphson?

No `app.py`, antes de chamar `newton_raphson()`, a função digitada pelo usuário é parseada com `sympy.sympify()`, que converte a string em expressão simbólica. Em seguida, `sympy.diff(expr, x)` calcula a derivada simbolicamente. Por fim, `sympy.lambdify(x, expr, 'numpy')` converte a expressão em função numérica que pode ser chamada com valores reais.

■ Como o sistema converte a string da função digitada em algo calculável?

O fluxo é: string do usuário → `sympy.sympify(texto)` (parser simbólico) → `sympy.lambdify(x, expr, 'numpy')` → função Python numérica. Se o texto tiver sintaxe inválida, o `sympify` lança uma exceção que é capturada e exibida como mensagem de erro antes de qualquer tentativa de cálculo.

■ Como o Clark Calc lê os dados da tela em tempo real?

Via function calling do Groq. Ao montar a requisição à API, o código passa a função `obter_ultimo_calculo` como uma tool disponível ao modelo. Quando o Llama decide que precisa dos dados, ele emite uma chamada a essa tool; o código a executa, serializa `st.session_state['ultimo_calculo']` em JSON e devolve ao modelo, que então formula a resposta.

■ O que acontece no código quando a Bissecção detecta intervalo inválido?

Na primeira linha da função `bissecacao()` em `raizes/bissecacao.py`: `if func(a) * func(b) >= 0: return None, 'A função não muda de sinal...'` — retorna `None` imediatamente. O `app.py` verifica se o resultado é `None` e exibe a mensagem de erro no lugar do resultado, sem prosseguir.

■ O que são os `__init__.py` dentro das pastas de métodos?

São arquivos que marcam a pasta como um pacote Python, permitindo importações como `from metodos_numericos.raizes.bissecacao import bissecacao`. Sem eles, o Python não reconhece a pasta como módulo e as importações falhariam. No projeto, os `__init__.py` estão vazios — servem apenas para essa finalidade estrutural.

■ Como o sistema armazena e exibe a tabela de iterações?

Cada função de método iterativo retorna uma lista ou dicionário com os dados de cada passo (número da iteração, valor de x , erro etc.). O `app.py` passa esses dados para `st.dataframe()` ou `st.table()` do Streamlit, que renderiza a tabela automaticamente. O mesmo objeto é salvo em `st.session_state` para o Clark Calc ler.

■ O que é o Teorema de Bolzano e como ele se aplica à Bissecção?

O Teorema de Bolzano garante que uma função contínua em $[a, b]$ tem ao menos uma raiz nesse intervalo se $f(a)$ e $f(b)$ têm sinais opostos. A Bissecção depende exatamente disso: se $f(a) \cdot f(b) \geq 0$, o sistema rejeita os dados imediatamente e retorna mensagem de erro.

■ Qual a diferença entre convergência linear, superlinear e quadrática?

Linear (Bissecção, Ponto Fixo): o erro reduz por fator constante a cada iteração — ex.: 50% por passo. Superlinear (Secantes, ordem $\approx 1,618$): reduz mais rápido que linearmente, mas não dobra. Quadrática (Newton-Raphson): o número de casas decimais corretas praticamente dobra a cada iteração. Para tolerância 10^{-10} : Newton costuma precisar de ~ 5 iterações; Bissecção pode precisar de ~ 20 .

■ O que é uma matriz singular e por que impede a solução?

Matriz singular tem determinante zero — não tem inversa. Geometricamente: o sistema é inconsistente (sem solução) ou indeterminado (infinitas soluções). Nos algoritmos diretos, isso aparece quando o pivô chega a zero durante a eliminação, tornando a divisão impossível. O sistema retorna None.

■ O que é dominância diagonal estrita e por que garante convergência do Jacobi?

Cada linha satisfaz $|a_{ii}| > \sum_{j \neq i} |a_{ij}|$. Sob essa condição, prova-se que o raio espectral da matriz de iteração é menor que 1 — condição suficiente para convergência. O sistema emite aviso no terminal se isso não for satisfeito, mas não bloqueia a execução.

■ O que são substituição progressiva e retroativa?

Progressiva (forward): sistema triangular inferior — resolve x_1 , depois x_2 usando x_1 , etc. Usada no passo $Ly = Pb$ da Fatoração LU. Retroativa (backward): sistema triangular superior — resolve x_n primeiro, depois x_{n-1} usando x_n , etc. Usada no Gauss e no passo $Ux = y$ da Fatoração LU.

■ O que é a matriz de permutação P na Fatoração LU?

P registra as trocas de linha feitas durante o pivoteamento. A decomposição real é $PA = LU$. Antes de resolver $Ly = b$, aplica-se P ao vetor: calcula-se Pb (permuta as entradas de b na mesma ordem das trocas). O `scipy.linalg.lu` faz isso automaticamente.

■ O que é ponto flutuante e por que causa erros numéricos?

Computadores representam números reais com precisão finita em binário — a maioria das frações decimais não tem representação exata (ex.: 0,1 em binário é periódico). Cada operação introduz pequenos erros de arredondamento que se acumulam. Dividir por números muito pequenos amplifica esses erros — daí o pivoteamento parcial no Gauss.

■ Qual a diferença entre Jacobi e Gauss-Seidel?

Jacobi: todas as variáveis são calculadas com valores da iteração anterior — atualizações independentes (poderiam ser paralelas). Gauss-Seidel: cada variável usa imediatamente os valores recém-calculados na mesma iteração — aproveita informação mais recente. Resultado: Gauss-Seidel geralmente converge em menos iterações para a mesma tolerância.

■ O que são os resíduos no ajuste de curvas?

Resíduos são as diferenças entre os valores observados e os preditos pela curva: $r_i = y_i - \hat{y}_i$. O critério dos mínimos quadrados minimiza $\sum r_i^2$. Resíduos grandes indicam modelo inadequado; resíduos com padrão (não aleatórios) indicam que o tipo de curva escolhido não representa os dados.

■ O que é overfitting no ajuste de curvas?

Quando o grau do polinômio é alto demais, ele passa por todos os pontos mas oscila absurdamente entre eles. Com n pontos, um polinômio de grau $n-1$ sempre interpola perfeitamente mas não generaliza. O cálculo termina sem erro, mas o modelo é inútil para previsão fora dos pontos dados.

■ O que é cálculo simbólico vs numérico? Quando cada um é usado no projeto?

Simbólico (SymPy): manipula expressões como fórmulas, sem arredondamento — deriva, simplifica. Numérico (NumPy/SciPy): opera com floats, muito mais rápido mas sujeito a erros de ponto flutuante. No SuperMath: SymPy é usado só para derivar a função no Newton-Raphson; todos os cálculos iterativos são numéricos.

■ Como verificar se a solução encontrada está correta?

Para raízes: substituir na função original e verificar $f(x) \approx 0$ dentro da tolerância. Para sistemas lineares: calcular o resíduo $r = b - A \cdot x$ e verificar $\|r\| \approx 0$. O SuperMath exibe o resultado e a tabela de iterações; o Clark Calc pode analisar a convergência e confirmar se o resultado é confiável.

■ O que acontece se a tolerância for muito alta ou muito baixa?

Muito alta (ex.: $1e-1$): o método para cedo com resultado impreciso — o critério é satisfeito antes de chegar perto da raiz real. Muito baixa (ex.: $1e-15$): pode nunca ser atingida por limitação de ponto flutuante — o método roda até `max_iter` sem convergir, mesmo que a solução numérica possível já tenha sido encontrada.

■ Por que o Newton-Raphson falha quando a derivada é zero?

A fórmula é $x_{n+1} = x_n - f(x_n)/f'(x_n)$. Se $f'(x_n) = 0$, há divisão por zero — a tangente é horizontal e não intercepta o eixo x em nenhum ponto finito. No código: `if abs(dfx) < 1e-12: return None, 'A derivada se aproximou de zero.'` — o sistema verifica antes da divisão e retorna erro.

■ Se um método iterativo atingiu `max_iter` sem convergir, o resultado está errado?

Não necessariamente. Se a diferença entre as últimas iterações ainda for grande, provavelmente está incorreto. Se estiver próxima da tolerância, pode ser aceitável. O correto é aumentar `max_iter`, verificar a condição de convergência da matriz, ou diminuir a tolerância para diagnóstico.

■ O que muda no código se a função tiver múltiplas raízes?

Nada — cada execução encontra uma raiz por vez, a mais próxima do chute inicial ou dentro do intervalo. Para encontrar outras raízes, o usuário deve mudar o intervalo ou x_n . O gráfico da função ajuda a identificar quantas raízes existem e onde estão antes de rodar o algoritmo.

■ Por que usaram Streamlit e não Flask/Django?

Streamlit foi escolhido pela velocidade de desenvolvimento: formulários, gráficos e layout responsivo com poucas linhas de Python. Flask e Django requerem HTML, CSS e roteamento — exagero para um projeto cujo foco são os algoritmos numéricos, não a infraestrutura web.

■ O Clark Calc faz os cálculos numéricos?

Não. Ele é um modelo de linguagem que lê e explica os resultados via function calling. Os cálculos são feitos pelas rotinas Python (NumPy/SciPy/SymPy). O Llama acessa `st.session_state['ultimo_calculo']` via `tools.py` e interpreta os dados em linguagem natural.

■ Por que usar `scipy.linalg.lu` em vez de implementar a Fatoração LU do zero?

A implementação do `scipy` usa LAPACK internamente — a mesma biblioteca de álgebra linear de software profissional como MATLAB. É otimizada, testada e numericamente estável. Reimplementar do zero introduziria riscos de bugs e instabilidade sem nenhum ganho prático para o escopo do projeto.

■ O que é a norma infinita usada como critério de parada em Jacobi e Gauss-Seidel?

$\|v\|_\infty = \max |v_i|$ — o maior valor absoluto entre as componentes do vetor. O critério de parada é $\|x_{\text{novo}} - x_{\text{ant}}\|_\infty / \|x_{\text{novo}}\|_\infty < \text{tolerância}$. Usar norma infinita garante que nenhuma componente individual ainda está mudando significativamente, não apenas que a média dos erros é pequena.

■ Qual a complexidade computacional do Gauss?

$O(n^3)$ — cúbico no tamanho do sistema. Dobrar a dimensão aumenta o tempo $\sim 8\times$. Para os sistemas pequenos do SuperMath isso é irrelevante (microsegundos), mas para sistemas de milhares de equações, métodos iterativos ou esparsos são preferíveis.

■ O que é um sistema inconsistente vs indeterminado?

Inconsistente: sem solução — equações se contradizem (ex.: $x+y=1$ e $x+y=2$). Resulta em pivô zero com $b \neq 0$ durante a eliminação. Indeterminado: infinitas soluções — equações redundantes (ex.: $x+y=1$ e $2x+2y=2$). Resulta em pivô zero com $b=0$. Em ambos os casos, o sistema retorna `None` com mensagem de erro.

■ Por que o Gauss-Seidel geralmente converge mais rápido que o Jacobi?

Porque aproveita informação mais recente dentro da mesma iteração. No Jacobi, o valor de x_i recém-calculado só é usado na próxima iteração. No Gauss-Seidel, ele já é usado para calcular x_{i+1} na mesma iteração — reduzindo o erro mais rápido. Analogia: Gauss-Seidel é como usar o resultado do cálculo anterior imediatamente; Jacobi espera terminar todos antes de atualizar.

7 RESUMO RÁPIDO — UMA LINHA POR MÉTODO

Bisseção	Divide o intervalo ao meio até achar a raiz
Ponto Fixo	Itera $x = g(x)$ até convergir — depende de $g(x)$ bem escolhida
Newton-Raphson	Usa a derivada para convergir rápido (quadrático)
Secantes	Newton sem derivada — usa dois pontos para estimá-la
Gauss c/ Pivoteamento	Triangulariza o sistema e resolve de trás pra frente
Fatoração LU	Decompõe $A = LU$ e resolve em duas substituições
Jacobi	Iterativo: atualiza tudo junto com valores da iteração anterior
Gauss-Seidel	Iterativo: usa valores recém-calculados na mesma iteração
Regressão Linear	Ajusta a melhor reta $y = ax + b$ pelo critério dos mínimos quadrados
Mínimos Quadrados	Ajusta o melhor polinômio de grau n aos dados

app.py	Interface Streamlit — importa e chama todos os métodos
tools.py	Função que serializa a sessão para o Clark Calc (function calling)
raizes/bissecao.py	Método da Bisseção
raizes/ponto_fixo.py	Método do Ponto Fixo
raizes/newton_raphson.py	Método de Newton-Raphson
raizes/secantes.py	Método das Secantes
sistemas/gauss_pivoteamento.py	Eliminação de Gauss com Pivoteamento
sistemas/fatoracao_lu.py	Fatoração LU (usa <code>scipy.linalg</code>)
sistemas/jacobi.py	Método de Jacobi
sistemas/gauss_seidel.py	Método de Gauss-Seidel
ajustes/regressao_linear.py	Regressão Linear
ajustes/minimos_quadrados.py	Mínimos Quadrados Polinomial (usa <code>np.polyfit</code>)

■ Dica: execute ao menos um cálculo de cada módulo antes da apresentação — facilita demonstrar o Clark Calc respondendo sobre um resultado real na tela.